

## Week8-RNN-2nd Lab

April 4, 2026

```
[43]: import glob
import os
import string
import unicodedata
import random
import math
import torch
import torch.nn as nn

DATA_PATH = "data/names"
HIDDEN_SIZE = 128
LEARNING_RATE = 0.005
N_ITERS = 100000
PRINT_EVERY = 5000
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

all_letters = string.ascii_letters
n_letters = len(all_letters)

def unicodeToAscii(s):
    return ''.join(
        c for c in unicodedata.normalize('NFD', s)
        if unicodedata.category(c) != 'Mn'
        and c in all_letters
    )

def load_data(path):
    category_lines = {}
    all_categories = []

    for filename in glob.glob(path + '/*.txt'):
        category = os.path.splitext(os.path.basename(filename))[0]
        all_categories.append(category)

        with open(filename, encoding='utf-8') as f:
            lines = f.read().strip().split('\n')
            lines = [unicodeToAscii(line) for line in lines]
```

```

        category_lines[category] = lines

    return category_lines, all_categories

category_lines, all_categories = load_data(DATA_PATH)
n_categories = len(all_categories)

print("Loaded languages:", n_categories)

def letterToIndex(letter):
    return all_letters.find(letter)

def lineToTensor(line):
    tensor = torch.zeros(len(line), 1, n_letters, device=DEVICE)
    for i, letter in enumerate(line):
        idx = letterToIndex(letter)
        if idx != -1:
            tensor[i][0][idx] = 1
    return tensor

class RNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()

        self.hidden_size = hidden_size

        self.i2h = nn.Linear(input_size + hidden_size, hidden_size)
        self.i2o = nn.Linear(input_size + hidden_size, output_size)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, input, hidden):
        combined = torch.cat((input, hidden), 1)

        hidden = self.i2h(combined)
        output = self.i2o(combined)
        output = self.softmax(output)

        return output, hidden

    def initHidden(self):
        return torch.zeros(1, self.hidden_size, device=DEVICE)

def randomTrainingExample():
    category = random.choice(all_categories)
    line = random.choice(category_lines[category])

    category_tensor = torch.tensor(

```

```

        [all_categories.index(category)],
        dtype=torch.long,
        device=DEVICE
    )

    line_tensor = lineToTensor(line)
    return category, line, category_tensor, line_tensor

rnn = RNN(n_letters, HIDDEN_SIZE, n_categories).to(DEVICE)
criterion = nn.NLLLoss()

def train(category_tensor, line_tensor):
    hidden = rnn.initHidden()
    rnn.zero_grad()

    for i in range(line_tensor.size(0)):
        output, hidden = rnn(line_tensor[i], hidden)

    loss = criterion(output, category_tensor)
    loss.backward()

    # manual SGD
    for p in rnn.parameters():
        p.data.add_(-p.grad.data, alpha=LEARNING_RATE)

    return output, loss.item()

start = time.time()

for i in range(1, N_ITERS + 1):
    category, line, category_tensor, line_tensor = randomTrainingExample()
    output, loss = train(category_tensor, line_tensor)

    if i % PRINT_EVERY == 0:
        guess_i = torch.argmax(output).item()
        guess = all_categories[guess_i]

        correct = " Correct " if guess == category else f"Wrong- Actual:␣
→({category})"

        print(f"{i} | Loss: {loss:.4f} | {line} → {guess} {correct}")

torch.save(rnn.state_dict(), "rnn_name_classifier.pth")

print("\nModel saved!")

def predict(input_line, n_predictions=3):

```

```

with torch.no_grad():
    line_tensor = lineToTensor(input_line)
    hidden = rnn.initHidden()

    for i in range(line_tensor.size(0)):
        output, hidden = rnn(line_tensor[i], hidden)

    topv, topi = output.topk(n_predictions, 1, True)

    predictions = []
    for i in range(n_predictions):
        value = topv[0][i].item()
        category_index = topi[0][i].item()
        predictions.append((all_categories[category_index], value))

    return predictions

print("\nPredictions:")
test_names = ["Schmidt", "Garcia", "Ivanov", "Kim"]
pred=["German","Portugese","Russian","Korean"]

for i in range(len(test_names)):
    name=test_names[i]
    preds = predict(name)
    print(f"\n{name}:")
    max_sc=float('-inf')
    max_lang=None
    for lang, score in preds:
        print(f"  {lang} ({score:.4f})")
        if (score>max_sc):
            max_sc=score
            max_lang=lang
    print("Predicted Language: ", lang)
    print("Actual Language: ", pred[i])

```

```

Loaded languages: 18
5000 | Loss: 2.8344 | ONeal → Arabic Wrong- Actual: (Irish)
10000 | Loss: 2.9035 | Pascal → Irish Wrong- Actual: (French)
15000 | Loss: 2.7007 | Desrosiers → Greek Wrong- Actual: (French)
20000 | Loss: 3.1116 | Santiago → Italian Wrong- Actual: (Portuguese)
25000 | Loss: 0.3166 | Ardovini → Italian Correct
30000 | Loss: 0.9454 | Torres → Portuguese Correct
35000 | Loss: 2.6349 | Silva → Czech Wrong- Actual: (Portuguese)
40000 | Loss: 2.8922 | Denis → Greek Wrong- Actual: (French)
45000 | Loss: 1.1824 | Han → Chinese Wrong- Actual: (Vietnamese)
50000 | Loss: 2.3568 | Aquino → Portuguese Wrong- Actual: (Spanish)
55000 | Loss: 0.9332 | Said → Arabic Correct

```

```
60000 | Loss: 0.1232 | Jidkov → Russian Correct
65000 | Loss: 2.4695 | Murphy → Scottish Wrong- Actual: (Irish)
70000 | Loss: 4.8461 | Can → Chinese Wrong- Actual: (Dutch)
75000 | Loss: 2.3513 | Shum → Korean Wrong- Actual: (Chinese)
80000 | Loss: 2.6515 | Arendonk → Italian Wrong- Actual: (Dutch)
85000 | Loss: 1.1386 | Nunez → Spanish Correct
90000 | Loss: 0.2156 | So → Korean Correct
95000 | Loss: 0.0291 | McGregor → Scottish Correct
100000 | Loss: 1.5523 | Pasternack → Czech Wrong- Actual: (Polish)
```

Model saved!

Predictions:

Schmidt:

English (-1.0008)

German (-1.6924)

French (-2.3024)

Predicted Language: French

Actual Language: German

Garcia:

Spanish (-0.8230)

Italian (-1.8866)

Portuguese (-2.0213)

Predicted Language: Portuguese

Actual Language: Portugese

Ivanov:

Russian (-0.0517)

Greek (-3.5120)

Czech (-4.9636)

Predicted Language: Czech

Actual Language: Russian

Kim:

Korean (-0.8876)

Chinese (-1.3524)

Vietnamese (-1.9766)

Predicted Language: Vietnamese

Actual Language: Korean

```
[31]: import torch
import torch.nn as nn
import random
import string
```

```

all_characters = string.printable
n_chars = len(all_characters)

HIDDEN_SIZE = 128
LEARNING_RATE = 0.005
N_ITERS = 20000
PRINT_EVERY = 2000
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

text = """hello world
deep learning"""

def charToIndex(c):
    return all_characters.find(c)

def charToTensor(c):
    tensor = torch.zeros(1, n_chars, device=DEVICE)
    tensor[0][charToIndex(c)] = 1
    return tensor

def lineToTensor(line):
    tensor = torch.zeros(len(line), 1, n_chars, device=DEVICE)
    for i, c in enumerate(line):
        tensor[i][0][charToIndex(c)] = 1
    return tensor

class RNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()

        self.hidden_size = hidden_size

        self.i2h = nn.Linear(input_size + hidden_size, hidden_size)
        self.i2o = nn.Linear(input_size + hidden_size, output_size)

    def forward(self, input, hidden):
        combined = torch.cat((input, hidden), 1)

        hidden = torch.tanh(self.i2h(combined))
        output = self.i2o(combined)

        return output, hidden

    def initHidden(self):
        return torch.zeros(1, self.hidden_size, device=DEVICE)

def randomTrainingExample():

```

```

start_index = random.randint(0, len(text) - 2)
end_index = start_index + random.randint(5, 15)

chunk = text[start_index:end_index]

input_seq = chunk[:-1]
target_seq = chunk[1:]

input_tensor = lineToTensor(input_seq)
target_tensor = torch.tensor(
    [charToIndex(c) for c in target_seq],
    dtype=torch.long,
    device=DEVICE
)

return input_tensor, target_tensor

rnn = RNN(n_chars, HIDDEN_SIZE, n_chars).to(DEVICE)
criterion = nn.CrossEntropyLoss()

def train(input_tensor, target_tensor):
    hidden = rnn.initHidden()
    rnn.zero_grad()

    loss = 0

    for i in range(input_tensor.size(0)):
        output, hidden = rnn(input_tensor[i], hidden)
        loss += criterion(output, target_tensor[i].unsqueeze(0))

    loss.backward()

    for p in rnn.parameters():
        p.data.add_(p.grad.data, alpha=-LEARNING_RATE)

    return loss.item() / input_tensor.size(0)

for i in range(1, N_ITERS + 1):
    input_tensor, target_tensor = randomTrainingExample()
    loss = train(input_tensor, target_tensor)

    if i % PRINT_EVERY == 0:
        print(f"Iter {i} | Loss: {loss:.4f}")

def generate(start_str="he", predict_len=100):
    with torch.no_grad():
        input_tensor = lineToTensor(start_str)

```

```

hidden = rnn.initHidden()

# feed initial string
for i in range(len(start_str) - 1):
    _, hidden = rnn(input_tensor[i], hidden)

last_char = input_tensor[-1]
output_str = start_str

for _ in range(predict_len):
    output, hidden = rnn(last_char, hidden)

    probs = torch.softmax(output, dim=1)
    topi = torch.multinomial(probs, 1)[0]

    predicted_char = all_characters[topi]
    output_str += predicted_char

    last_char = charToTensor(predicted_char)

return output_str

print("\nGenerated Text:\n")
print(generate("deep ", 8))
print(generate("hello ", 5))
print(generate("world ", 5))

```

```

Iter 2000 | Loss: 0.5922
Iter 4000 | Loss: 0.4539
Iter 6000 | Loss: 0.4312
Iter 8000 | Loss: 0.3356
Iter 10000 | Loss: 0.0883
Iter 12000 | Loss: 0.0831
Iter 14000 | Loss: 0.1931
Iter 16000 | Loss: 0.2563
Iter 18000 | Loss: 0.0877
Iter 20000 | Loss: 0.3968

```

Generated Text:

```

deep learning
hello world
world deep

```

[ ]: